Zone: Extending a bin2bin Metamorphic Engine with Code–Data Identification and LLM Assistance

Stefano Bistarelli[0000−0001−7411−9678], Jacopo Di Benedetto, and Francesco Santini[0000−0002−3935−4696]

Dipartimento di Matematica e Informatica, Università degli Studi di Perugia,
Via Vanvitelli 1, 06123 Perugia (PG), Italy
stefano.bistarelli@unipg.it
jacopo.dibenedetto@studenti.unipg.it
francesco.santini@unipg.it

Abstract. Zone is a binary-to-binary metamorphic engine for compact Windows x86 and x64 payloads. Its instruction substitution, insertion, and control-flow transformations include the Position-Independent Instruction representation introduced in the conference version of this work. This extended article retains the original design and empirical study of 102 inputs under fifteen configurations, assessed through static-scan verdicts, functional checks, size, entropy, and similarity. The published observations show that changes in detectability and functional preservation must be evaluated separately. Building on the code–data ambiguity identified in that study, we add a data-handling layer with symbolic and heuristic identification and an alternative LLM-based classification path. We also describe LLM-assisted assembly transformation with zero-shot and two-shot conditions and integrate assembly wrapping as an existing technique, with its x86 port under development. A separate assessment protocol addresses classification quality, construction success, working rate, static-scan measurements, representation similarity, output diversity, and cost. The original results provide the empirical foundation for the extension, while new campaigns are identified independently. Conclusions remain bounded by the input scope, observation protocol, and raw-binary scan setting.

Keywords: Binary Rewriting · Metamorphic Code · Code–Data Identification · Large Language Models · Functional Preservation

Draft note. The original study and its detailed results are retained. Additional experiments are in progress; their numerical results remain pending. The x86 wrapping port is development work. Update the abstract and conclusions with the verified extension findings before submission.

1 Introduction

Code transformations complicate the analysis of malicious software because distinct binary representations may implement the same behavior. Metamorphism is one form of transformation-based evasion [12]: the program's instruction sequence is rewritten, whereas polymorphic encoding typically changes an encrypted representation and its

decoding routine. Both approaches challenge byte-pattern recognition, but they have different implications for the representation available during execution [5,20].

Zone provides a modular environment for examining these effects on compiled Windows x86 and x64 payloads. Its *bin2bin* interface accepts binary input without source code or debugging symbols. The transformations considered here are equal code substitution (EQ), NOP insertion, instruction block permutation (IBP), bogus control flow (BCF), and position-independent instruction (PII). Their individual and combined effects are assessed through detection measurements, functional tests, and changes in size and instruction statistics.

Relationship to the Conference Paper and Extensions. This article extends our SECRYPT 2026 study [4]. It retains the engine description, PII design, original experimental protocol, detailed results, and comparison with Shikata Ga Nai as the foundation of the extended investigation. Reused figures, tables, listings, and the algorithm carry source citations. Additional sections address the limitations identified by that study through:

- symbolic and heuristic code–data identification, an alternative LLM-based classifier, and shared deterministic data handling;
- LLM-assisted assembly transformation with zero-shot and two-shot conditions, validation, and feedback;
- integration and assessment of assembly wrapping, an existing technique, with the x86 implementation under development;
- additional evaluation of classification quality, build and functional outcomes, static verdicts, similarity, diversity, and cost.

The extension complements the original experiments. Its methods and results are identified separately so that the published findings remain available for comparison and are not conflated with new measurements.

The scope is restricted to compact, position-independent payloads rather than complete Portable Executable (PE) files. Full-file rewriting requires additional treatment of section boundaries, imports, relocation information, alignment, and entry points. Those format-dependent issues would complicate the interpretation of experiments intended to compare instruction-level transformations. Signatures in non-code regions of PE files are an additional consideration [22].

Operating on largely executable instruction sequences makes it possible to examine a substantial fraction of each input. It also limits the conclusions: these experiments address a payload-level representation, not the complete delivery and execution of a malicious application. In particular, functional preservation and the detectability of a raw binary must be considered separately.

The published baseline comprises 102 Windows payloads, including 38 x64 and 64 x86 inputs, evaluated under fifteen transformation configurations. Its main observation is a trade-off between changes in the binary representation and the proportion of variants that remain functional. Some configurations received no malicious verdicts in the VirusTotal scans, but such observations are specific to the file format and testing conditions; they do not establish general resistance to deployed endpoint products.

Sections 2 and 3 retain the background and engine design. The original experiments and representation measurements appear in Sections 4 and 5, followed by their limitations and the SGN comparison (Sections 6 and 7). Sections 8, 9, and 10 describe the additional methods. Sections 11 and 12 define and reserve the extended assessment. Section 13 discusses related work and Section 14 concludes.

2 Background

The following background distinguishes metamorphic rewriting from polymorphic encoding and introduces the transformation families used in the baseline evaluation.

2.1 Metamorphic Engines

A metamorphic engine constructs a different instruction-level representation of an input program intended to preserve its behavior. Early virus generators used this idea to vary their byte sequences between instances [25]. Functional preservation is a design objective and must be checked; it does not follow merely from the fact that a transformation has produced an executable output.

A key difference is the absence of a mandatory decrypt-and-execute stage [29]. An encrypted payload is normally recovered before execution, while a successfully rewritten payload can execute in its transformed representation. This changes what a memory scan may observe, without implying that the transformed program is undetectable.

Instruction substitution, insertion, and reordering can generate many representations from the same input [29,17]. This diversity complicates the use of fixed byte patterns, and places a substantial burden on the rewriting engine to preserve control flow, data references, and processor state. Transformation-based evasion and its limitations are surveyed in [12].

Comparison with Polymorphism. Polymorphic encoding usually combines an encoded body with a variable decoding routine [25]. Shikata Ga Nai is a representative example: variation in its decoding stub changes the static byte sequence, but recurring features of that stub can themselves be used for recognition [18]. Once decoding has completed, the original body may be available to memory analysis. Metamorphic rewriting instead changes the body that is executed [11]; this is a distinction between representations, not a guarantee of runtime evasion.

2.2 Morphing Techniques

The established transformation families considered in Zone are EQ, NOP insertion, IBP, and BCF. They can be evaluated separately or in selected combinations. Their antecedents are discussed in Section 13.

EQ changes the instruction vocabulary; NOP insertion adds operations intended to leave the relevant computation unchanged. IBP modifies the physical layout of instruction blocks, whereas BCF introduces branches and unreachable regions. PII, described in Section 3.1, further separates instruction placement from logical execution order. These descriptions summarize the design reported in [4].

Table 1. Instruction substitutions and corresponding encodings (x86). Reproduced from Table 1 of [4].

Original Inst.	Morphed Inst.	Orig. Bytes	Morphed Bytes
`XOR eax, eax`	`MOV eax, 0`	`31 C0`	`B8 00 00 00 00`
`SUB eax, 1`	`ADD eax, -1`	`83 E8 01`	`83 C0 FF`
`ADD eax, 2`	`SUB eax, -2`	`83 C0 02`	`83 E8 FE`
`MOV eax, 0`	`PUSH 0; POP eax`	`C7 C0 00..`	`6A 00 58`

Table 2. NOP-like operations and their encodings. Reproduced from Table 2 of [4].

Example	Compiled Opcode
`NOP`	`0x90`
`SUB rax, 0x0`	`0x48, 0x83, 0xE8, 0x00`
`ADD rax, 0x0`	`0x48, 0x83, 0xC0, 0x00`
`MOV rax, rax`	`0x48, 0x89, 0xC0`
`LEA rax, [rax + 0]`	`0x48, 0x8D, 0x40, 0x00`
`PUSH rax; POP rax`	`0x50, 0x58`
`OR rax, rax`	`0x48, 0x09, 0xC0`
`XOR rax, 0x0`	`0x48, 0x83, 0xF0, 0x00`
`AND rax, rax`	`0x48, 0x21, 0xC0`
`INC rax; DEC rax`	`0x48, 0xFF, 0xC0; 0x48, 0xFF, 0xC8`
`DEC rax; INC rax`	`0x48, 0xFF, 0xC8; 0x48, 0xFF, 0xC0`
`SAR rax, 0x0`	`0x48, 0xC1, 0xF8, 0x00`
`SHR rax, 0x0`	`0x48, 0xC1, 0xE8, 0x00`
`SHL rax, 0x0`	`0x48, 0xC1, 0xE0, 0x00`
`RCL rax, 0x0`	`0x48, 0xC1, 0xD0, 0x00`
`RCR rax, 0x0`	`0x48, 0xC1, 0xD8, 0x00`

Equal Code Substitution. EQ replaces an operation with an alternative representation intended to have the same effect. For example, the zeroing operations `XOR rax, rax` and `MOV rax, 0x0` have different encodings. Such examples illustrate changes in byte representation; preservation of all relevant state remains an implementation concern.

Table 1 collects the substitution examples used in the conference study, including arithmetic complements and stack-based sequences. Instruction substitution and its use with insertion transformations have a long history [7,8,23].

NOP/Dead Code Insertion. Insertion transformations introduce operations intended to leave the program's relevant behavior unchanged. The single-byte `NOP` is one example; Table 2 reproduces the other examples considered in the original implementation.

Some apparent no-ops, including arithmetic with zero, can change processor flags. The original implementation therefore used flag-preserving sequences around such operations, as illustrated in Table 3. The resulting repeated patterns are themselves relevant to detection: changes introduced by an engine can create recognizable regularities [3].

Table 3. Flag-preserving insertion example. Reproduced from Table 3 of [4].

Instruction	Compiled Opcode
PUSHF	0x9C
ADD rax, 0x0	0x48, 0x83, 0xC0, 0x00
POPF	0x9D

Table 4. Illustrative BCF block. Reproduced from Table 4 of [4].

Address	Instruction
0x0	mov rax, 0x1
0x7	cmp rax, 0x1
0xb	je 0x18
0x11	nop
0x12	nop
0x13	nop
0x14	nop
0x15	nop
0x16	nop
0x17	nop
0x18	original code.....

Instruction Block Permutation. IBP changes where instruction blocks are stored while retaining their intended logical succession. The original implementation organizes the transformation into the following stages:

1. **Partitioning.** The instruction sequence is divided into blocks; the chosen granularity affects both rewriting overhead and the amount of layout variation.
2. **Connecting blocks.** An unconditional branch links each block to its logical successor.
3. **Changing placement.** Blocks are permuted, while the entry block retains its position.
4. **Updating references.** Control-flow targets are adjusted to the resulting layout.

This transformation changes adjacency without necessarily changing most instructions within a block. Consequently, a detector based on a short pattern internal to an unchanged region may behave differently from one that relies on a longer contiguous sequence. The experiment assesses these changes empirically rather than assuming that every signature is disrupted.

Bogus Control Flow. BCF adds branches around code that is not intended to execute. The implementation uses conditions with predetermined outcomes to exclude the inserted region from the execution path [10,15]. Table 4 reproduces the illustrative block from the conference article: the condition transfers control past the padding at addresses `0x11`–`0x17`.

1. **Inserted regions.** The engine creates the additional blocks, which may contain instruction sequences or raw bytes.

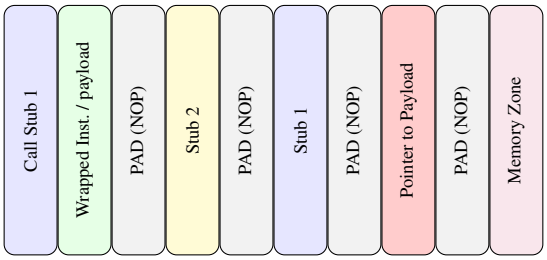


Fig. 1. PII payload layout. Adapted from Fig. 1 of [4].

2. **Predetermined conditions.** Conditions are constructed to select a known branch outcome.
3. **Integration.** The conditions and corresponding branches are added to the instruction stream. Table 4 illustrates the resulting organization.

3 Zone Architecture

The name Zone refers to Tarkovsky's *Stalker* and its alternative paths toward a common destination. The open-source implementation[1] was originally explored for both complete PE files and raw payloads. The extended manuscript retains the payload-level scope of the conference paper [4]; the difficulties of maintaining PE metadata are not evaluated as a separate contribution.

The implementation is written in Python. Capstone disassembles the input, the selected transformations modify the instruction representation, and Keystone assembles the output. The *bin2bin* interface requires neither source code nor Program Database (PDB) symbol files.

3.1 Position-Independent Instruction

PII was introduced in the conference paper [4]. It decouples the physical position of an instruction from its place in the intended execution sequence.

Organization. Consider an instruction such as `ADD rax, 1`. In the PII representation it is placed in a wrapper rather than relying on adjacency to its successor. The wrapper executes the original operation and transfers control to a redirector. An offset table supplies the location of the next logical instruction. Figure 1 reproduces the overall layout, while the listings below document the components of the original design.

The representation consists of four components:

1. **Base-Address Stub (Stub1).** This component determines the runtime base address and records it in a designated memory location.

[1] Project repository: https://github.com/jakydibe/Zone.

2. **Redirector Stub (Stub2).** This component obtains the next logical instruction location from the offset table and dispatches execution to it.
3. **Offset Table.** This contiguous table contains the instruction offsets used by the redirector.
4. **Wrapped Instructions.** Each input instruction is placed in a wrapper intended to maintain the required register, flag, and stack state.

Execution Sequence. The conference paper describes execution in three phases:

– **Initialization.** A call enters Stub1 (Listing 1). The return address supplied by that call is used to establish the payload's runtime location.
– **Original operation.** The wrapper in Listing 2 restores the saved state, adjusts the stack, executes the input instruction, and supplies its logical index to Stub2.
– **Dispatch.** The redirector in Listing 3 saves the state it uses, obtains the successor's location from the offset table and base address, and transfers control to that wrapper after restoring the relevant state.

Physical Placement. The original implementation permutes complete wrappers, treating each five-instruction wrapper as a unit. The offset table is updated for the resulting placement. Execution still starts with the initial call to Stub1; logical succession is represented by the table rather than by the wrappers' physical adjacency.

Listing 1. Base-address component. Reproduced from Listing 1 of [4].

```
pop <random_unused_reg>;    popping 2nd instruction address
mov [<random_unused_reg> - 5 + <init_address>],  <random_unused_reg>; move it to a memory
     slot
push <random_unused_reg>; push just to avoid altering the stack
jmp <random_unused_reg>; return to the caller (2nd instruction)
```

Listing 2. Instruction wrapper. Reproduced from Listing 2 of [4].

```
pop rax;                    ; popping rax because we just used it to save jmp address
lea rsp, [rsp + 8];         ; adjusting stack to avoid stack overflow
ORIGINAL_INSTRUCTION;       ; original instruction
push <0x3e8 + counter>;     ; pushing instruction original index + a constant
jmp <stub2_address>;        ; Jumping to 'redirector' stub
```

Listing 3. x64 redirector. Reproduced from Listing 3 of [4].

```
push rax;                                       saving original 'rax' register state
push rcx;                                       saving original 'rcx' register state
mov rcx, [rsp + 0x10] ;                         getting the instruction index from the stack.
pushf;                                          saving original CPU flags
sub rcx, 0x3e8; ;                               subtracting a constant to get instr index
inc rcx;                                        increase instruction number
mov rax, [rip + init_addr_offset];              get payload base address
mov rcx, [rax + mem_address - 5 + rcx * 8];     get next instruction address
sub rcx, 0x5;                                   just because base address is 2nd instruction
add rax, rcx;                                   add payload base address to the offset
popf;                                           restore cpu flags
pop rcx;                                        restore rcx value
jmp rax;                                        jmp to next instruction
```

Architectural Variants. Listings 3 and 4 show the published x64 and x86 redirectors. In addition to the different register widths, they differ in how the required address is obtained: the x86 version uses the call-based mechanism documented in the original listing rather than x64 RIP-relative addressing.

Listing 4. x86 redirector. Reproduced from Listing 4 of [4].

```
push eax;
push ecx;
mov ecx, [esp + 0x8]
pushf;
sub ecx, 0x3e8;
inc ecx;

call <stub2_address + 0x14>;    calling pop eax, 0x14 is pop eax offset from stub2 address
pop eax;                        popping eax puts in eax the value of the instruction pointer

mov eax, [eax + <init_addr_offset>];
mov ecx, [eax + <mem_address> - 5 + ecx * 4];
sub ecx, 0x5
add eax, ecx
popf;
pop ecx;
jmp eax;
```

3.2 How Instructions Are Patched

Moving instructions or inserting bytes can invalidate references to their previous locations. The patching phase updates control-flow destinations and other affected references to the modified layout. Branches and calls are therefore part of the rewriting process, not merely instructions that can be copied without regard to their new positions.

This requirement also explains the scope restriction. Rewriting raw instruction sequences already requires consistent target updates; complete PE files introduce additional metadata and layout dependencies. Algorithm 1 reproduces the high-level workflow from [4].

3.3 Transformation Pipeline

The original workflow in Algorithm 1 can be read as six stages. The additional analysis and transformation paths are described in Sections 8–10.

1. **Input representation.** Capstone produces instruction objects held in a linked sequence, with an address-indexed dictionary for lookup (Line 4).
2. **Embedded data.** The implementation enables Capstone's `Skipdata` option (Line 5). This does not solve the general problem of identifying inline data, particularly when data bytes also admit an instruction interpretation [10].
3. **Reference tracking.** A label table associates each tracked control-flow operation with its target instruction object. This association is retained as the object's physical address changes.
4. **Transformation and layout update.** The chosen transformations are applied (Line 19), followed by recomputation of instruction addresses (Line 12).
5. **Reassembly of tracked branches.** Keystone assembles the tracked control-flow operations using their updated targets (Line 25).
6. **Output.** The resulting instruction bytes are written as a new raw binary (Line 36).

Algorithm 1: High-level Zone workflow. Reproduced from Algorithm 1 of [4].

```
Data: raw_bytes
Result: Transformed binary
global instruction_list ← [];
global instruction_dict ← {};
global label_table ← [];
Function disass
    CAPSTONE.SKIPDATA ← TRUE;
    for each instruction in disasm(raw_bytes)
        instruction_list.append(instruction);
        instruction_dict[instruction.address] ← instruction;
        if instruction is X86.Control_flow_instruction
            jump_address ← extract_address(instruction.op_string);
            label_table.append((instruction, instruction_dict[jump_address]));
Function update_instructions
    new_instruction_dict ← {};
    for each instruction in instruction_list
        address ← instruction.previous_instruction.address + len(instruction.previous_instruction.bytes);
        instruction.address ← address;
        new_instruction_dict[instruction.address] ← instruction;
    instruction_dict ← new_instruction_dict;
Function morph
    equal_code_substitution();
    nop_insertion();
    instruction_block_shuffling();
    bogus_control_flow();
    position_independent_instruction();
Function adjust_label_table
    update_instructions();
    for each entry in label_table
        jmp_addr ← extract_address(entry[0].op_string);
        jmp_addr_new ← entry[1].address;
        new_instruction ← keystone.assemble( “entry[0].mnemonic jmp_addr_new”);
        entry[0] ← new_instruction;
Function main
    disass();
    morph();
    adjust_label_table();
    write_to_file();
```

4 Published Experimental Baseline

Draft note. The detailed historical plots and their reported numerical values are retained from the preparation manuscript. Some category/value associations in the detection and working-rate plots, and the provenance of the similarity metric, require reconciliation against the conference version and archived records. These are baseline editorial checks, not replacement experiments. See `BASELINE_CHECKS.md`.

This section restates the protocol and results of the conference paper [4]. It introduces no additional measurements. VirusTotal scans supplied verdicts from more than 70 engines, subject to the file-format limitations discussed below.

The corpus contains 102 Windows payloads: 38 x64 inputs and 64 x86 inputs. Most originate from the Metasploit Framework and include reverse and bind shells. This pro-

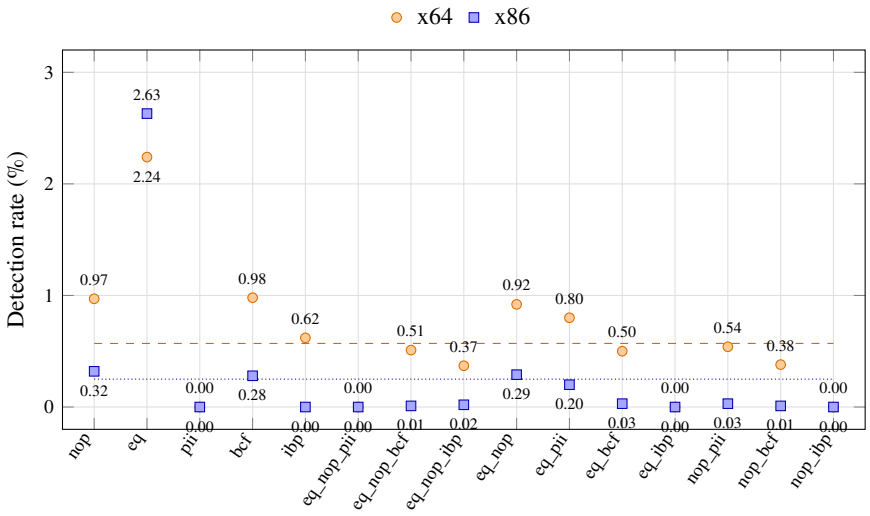


Fig. 2. Malicious-verdict fractions in raw-file scans. Dashed reference lines show the reported architecture averages (x64: 0.57%; x86: 0.25%). Data from Fig. 2 of [4]; redrawn for the present format.

vides an identifiable collection of payload types, but does not constitute a representative sample of all Windows malware.

Two scope restrictions are relevant to interpretation:

- **Architecture coverage.** The x64 and x86 sets differ in size and membership. Architecture-level comparisons therefore also reflect differences between the two input collections.
- **Excluded inputs.** Large, DLL-based command-and-control components are outside the payload-level scope. Their PE structure and embedded data introduce additional rewriting dependencies.

Testing Method. Each of the fifteen configurations was applied to every input, with ten generated variants per input and configuration to expose variation in the transformation process. The five individual transformations and ten combinations were:

- **Single:** NOP (`nop`), Equal Code Substitution (`eq`), Bogus Control Flow (`bcf`), Instruction Block Permutation (`ibp`), and Position-Independent Instruction (`pii`).
- **Combined:** `eq_nop_bcf`, `eq_nop_ibp`, `eq_nop_pii`, `eq_nop`, `eq_bcf`, `eq_ibp`, `eq_pii`, `nop_bcf`, `nop_ibp` and `nop_pii`.

The *detection rate* for an individual file is the fraction of scanning engines that returned a malicious verdict; summaries aggregate these per-file measurements. Figure 2 presents the published transformed-file results. The corresponding baseline averages for the original files were 21.67% for x64 and 22.63% for x86.

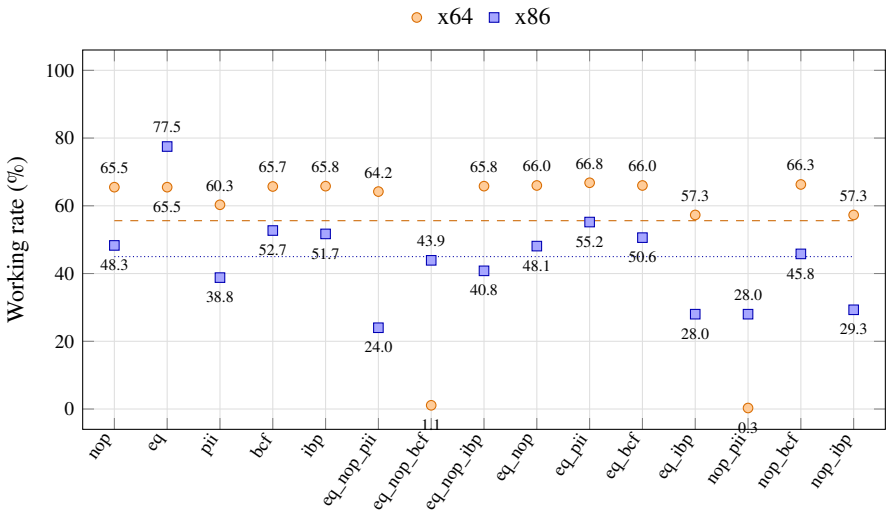


Fig. 3. Variants passing the published functional checks. Dashed reference lines show the reported architecture averages (x64: 55.6%; x86: 45.0%). Data from Fig. 3 of [4]; redrawn for the present format.

The plotted observations vary by transformation and architecture. Several configurations involving PII received no malicious verdicts in these raw-file scans, whereas EQ retained more detections. These measurements describe the tested representations and submission conditions; they should be interpreted together with the functional results.

The *working rate* is the proportion of variants that passed the original execution checks (Figure 3). Those checks observed the expected connection for reverse shells, port opening for bind shells, and exception-free termination for the remaining generic cases. The tests are empirical functional checks rather than proofs of semantic equivalence. The conference paper reported greater overall stability for EQ and lower stability for PII, particularly in the x86 collection; the results also varied substantially across combined configurations.

Evaluation Limitations. A raw `.bin` submission is not a complete executable application. Engines may apply a different analysis path to such input, omit execution-based analysis, or decline to analyze an unsupported representation. Low detection fractions therefore cannot be interpreted as an evaluation of complete antivirus or endpoint detection and response (EDR) products during deployment.

Submitting original and transformed payloads under the same format assumptions provides a limited basis for comparing the representations in this corpus. It does not remove format-dependent biases or establish behavior under other execution conditions. The published results should be read within those boundaries [4].

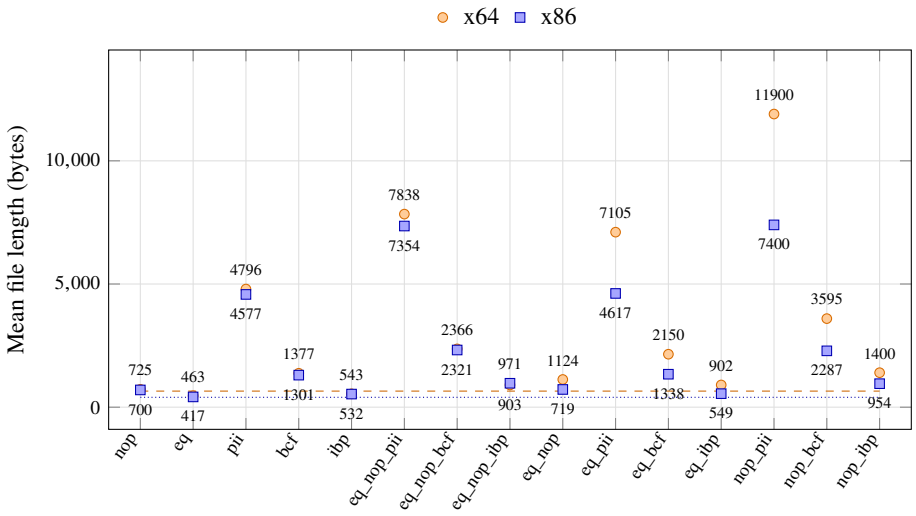


Fig. 4. Mean transformed-file length. Reference lines indicate the original-file means (x64: 652.632 bytes; x86: 400.750 bytes). Data from Fig. 4 of [4]; redrawn for the present format.

5 Size and Statistical Measurements

The conference paper also characterized how the transformed representations differ from their inputs [4]. Its three additional measurements were:

- Payload length variation.
- Delta entropy.
- Cosine similarity.

Payload Length Variation. Let L_i denote the byte length of a transformed file and let n be the number of files considered for a configuration. The reported mean length is:

$$\overline{L} = \frac{1}{n} \sum_{i=1}^{n} L_i$$

The size results in Figure 4 show substantial growth for PII and BCF configurations. Some EQ-containing configurations instead produced smaller average outputs than the original-file reference. The conference paper associated this observation with the implementation's treatment of data during disassembly; the size measurement alone does not establish that any removed bytes were semantically irrelevant.

Delta Entropy. The reported entropy change compares the transformed representation, H_{tech}, with its original, H_{orig}, using the percentage difference:

$$\Delta H\ (\%) = \frac{H_{tech} - H_{orig}}{H_{orig}} \times 100 \tag{1}$$

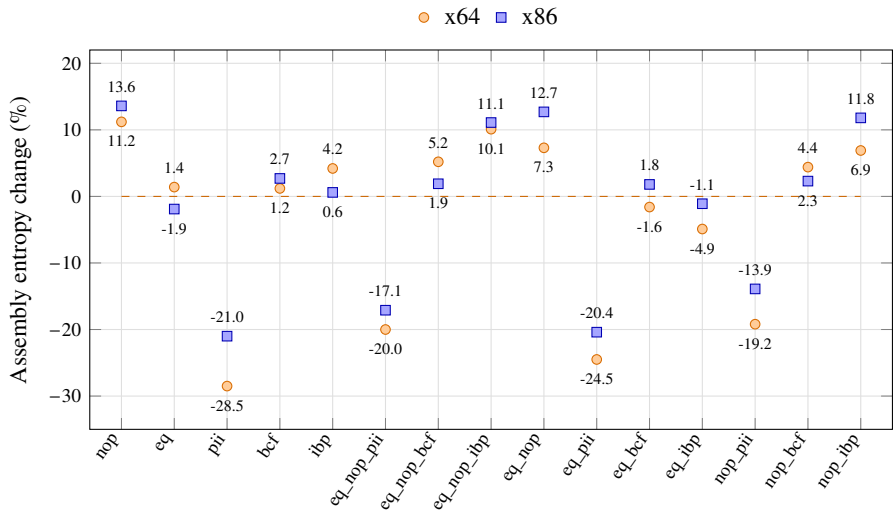


Fig. 5. Percentage entropy change for tokenized assembly; the reference level is zero. Data from Fig. 5 of [4]; redrawn for the present format.

The measurements were computed on raw bytes and on tokenized assembly. Figures 5 and 6 retain the published results. The assembly-based representation also permits comparison with the metrics discussed by Mohseni et al. [19].

In the assembly results, NOP insertion increased entropy for both architectures, while PII decreased it. The latter observation is consistent with the repeated bookkeeping instructions in the wrapper representation. A small entropy change does not, by itself, imply a small change in control-flow organization. The conference paper illustrated this limitation using BCF and IBP:

- **BCF.** A small entropy difference coexisted with greater length and a changed opcode distribution (Figure 7).
- **IBP.** Moving blocks can change their layout while retaining much of their instruction-frequency distribution.

Entropy is consequently treated as a descriptive statistic of the chosen representation, not a standalone measure of structural complexity or functional preservation.

Cosine Similarity. The frequency vectors A and B represent the opcode distributions of an original file and its transformed variant. Their cosine similarity is:

$$\mathrm{sim}(A,B) = \frac{A \cdot B}{\|A\|\,\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\,\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Here A_i and B_i are the frequencies of opcode i, and n is the size of their joint vocabulary. The score summarizes agreement between the two frequency profiles. Its use

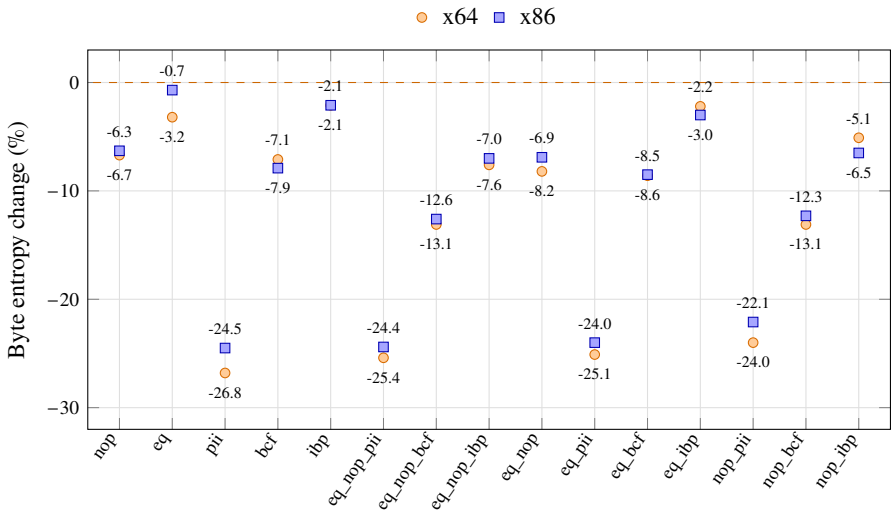


Fig. 6. Percentage entropy change for raw bytes; the reference level is zero. Data from Fig. 6 of [4]; redrawn for the present format.

in assembly-obfuscation evaluation is discussed in [19]; it does not establish that the programs have the same behavior.

Instruction-frequency agreement is insufficient to validate execution: a small error can change a program's behavior while leaving that distribution almost unchanged. For this reason, the baseline used the execution checks described in Section 4 to assess functionality separately.

Figure 7 shows higher similarity for EQ and IBP than for the insertion-heavy configurations. This is consistent with their different effects on the instruction vocabulary. The larger size changes in Figure 4 provide complementary information about the additional instructions; neither measurement should be substituted for a functional test.

6 Limitations and Defensive Implications

The following limitations concern the original implementation and experiments [4]. Sections 8–10 describe subsequent work motivated by these observations; the residual limits of the added paths are discussed in Section 11.4.

- **Instruction coverage.** Less common instructions, including the example `JRCXZ`, can require handling not exercised by this corpus. Testing a fixed collection of 102 inputs does not establish complete x86/x64 coverage.
- **Code–data ambiguity.** Embedded strings may partly decode as instructions. Table 5 illustrates an input in which the bytes of `.exe` are interpreted as a branch. Rewriting the resulting instruction interpretation can corrupt the embedded string.

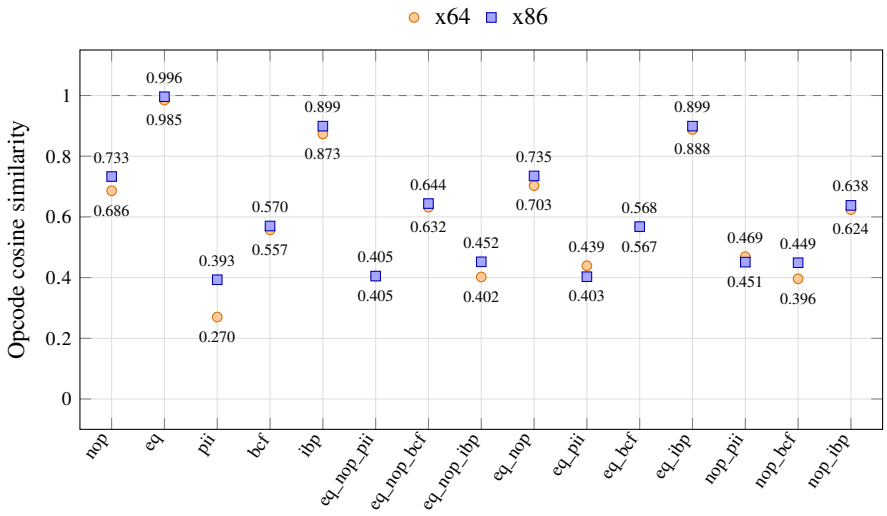


Fig. 7. Cosine similarity between original and transformed opcode-frequency vectors; the reference level is one. Data from Fig. 7 of [4]; redrawn for the present format.

Table 5. Inline-data example affected by rewriting. Reproduced from Table 5 of [4].

Bytes	Instruction
`0x109 b'\xff\xd5'`	call rbp
`0x10b b'c'`	.byte 0x63
`0x10c b'a'`	.byte 0x61
`0x10d b'l'`	insb byte ptr [rdi], dx
`0x10e b'c'`	.byte 0x63
`0x10f b'.exe'`	js 0x178
`0x113 b'\x00'`	.byte 0x00

Accurate recovery of instructions and references is a broader difficulty in static binary analysis [28]. General limitations of disassembly do not establish that perfect results are impossible on a bounded test corpus; they also do not excuse uncharacterized implementation failures. The reported working rates describe this implementation and these inputs.

The two examples below document the observed code–data distinction. They illustrate one failure mechanism, not a general guarantee about other payloads.

In Table 5, `calc.exe` is stored at the end of the input. Although some characters are represented as data, part of the string receives an instruction interpretation. This explains why the original implementation could modify bytes that were meant to remain data.

Table 6 shows the other published example, in which the string occurs as an immediate operand at address `0x8e`. The reported transformation did not encounter the same inline-data failure. This observation is specific to that example and does not imply general immunity from rewriting errors.

Table 6. Immediate-operand example. Reproduced from Table 6 of [4].

Bytes	Instruction
`0x8d b'Q'`	push rcx
`0x8e b'H\xb9calc.exe'`	movabs rcx, 0x6578652e636c6163
`0x98 b'Q'`	push rcx
`0x99 b'H\x8d\x0c\$'`	lea rcx, [rsp]
`0x9d b'H1\xd2'`	xor rdx, rdx
`0xa0 b'H\xff\xc2'`	inc rdx
`0xa3 b'H\x83\xec('`	sub rsp, 0x28
`0xa7 b'\xff\xd0'`	call rax

6.1 Mitigations and Countermeasures

Changing the binary representation does not remove a program's observable actions. Defensive analysis can consider memory protections, execution patterns, API activity, and network behavior in addition to static bytes. Recognition of a transformation engine is another possibility: repeated artifacts of a common toolchain can persist across outputs, as documented for Shikata Ga Nai [18]. Statistical or learned detectors must also account for benign programs that share similar features. These are defensive considerations arising from the study, not results of an additional detection experiment.

7 Comparison with Shikata Ga Nai

The conference paper [4] compared Zone with the Shikata Ga Nai (SGN) polymorphic encoder [13]. SGN combines additive-feedback encoding with a variable decoding stub. The comparison included the original x86 encoder and an unofficial x64 implementation [9]; recognition of the x86 stub has been discussed in [18].

The published aggregate static-scan observations were:

- **x64:** SGN achieved 0%, while Zone averaged 0.57% (reaching 0% with the PII technique).
- **x86:** SGN had a 10.22% detection rate, while Zone achieved 0.25% (also reaching 0% with PII).

The encoders also differ in the representation available after decoding. The conference paper submitted memory dumps obtained during SGN execution to VirusTotal and reported verdicts matching those for the unencoded inputs [4]. This observation is consistent with exposure of the original representation after decoding and with prior work on extracting attack code from memory [11].

Zone's intended execution model retains the rewritten representation. The dump experiment nevertheless measures the recognizability of submitted memory contents; it is not a direct measurement of an endpoint product's runtime detection. Claims about deployed EDR behavior require evidence beyond this comparison.

8 Code–Data Identification and Data-Preserving Rewriting

8.1 Problem Definition

Let an input payload be a byte sequence $B = (b_0, \ldots, b_{n-1})$. The identification task returns candidate data intervals $[s_j, e_j)$ in payload-relative coordinates, optionally accompanied by a kind such as an ASCII string, a UTF-16LE string, or another data object. A candidate identifies a proposed role for the bytes. Its acceptance for rewriting additionally depends on the available reference evidence and the capabilities of the data-handling layer.

The original inline-data failure in Table 5 and the immediate-operand example in Table 6 motivate this extension. Both examples and their interpretation remain in Section 6; the added layer explicitly represents candidate data and the references affected by rewriting.

The task also includes uncertainty. A region outside the recovered graph may contain data, unobserved code, or a mixture of the two. The pipeline retains the origin of a candidate rather than collapsing all such regions into the same evidential category. This permits separate assessment of candidate generation, filtering, and downstream outcomes.

8.2 Symbolic and Heuristic Identification

The non-LLM path combines static graph information with bounded symbolic exploration. A CFGFast pass supplies candidate code blocks and inferred data objects. Symbolic exploration records instruction addresses visited under the raw-payload model. An additional graph-recovery pass can use newly observed entries, so that dynamically observed code is not lost merely because it was absent from the first graph.

The symbolic initial state models uninitialized registers and memory symbolically. It does not initialize a complete Windows process or implement all external API behavior. Exploration is bounded by a time budget, a state/block-step limit, a pending-state limit, and solver budgets. Reports record the stop reason and unresolved states. Reaching a resource limit, encountering an unsupported operation, and exhausting the current frontier have different meanings and are reported separately.

The implementation maintains evidence at byte granularity by expanding observed instruction starts into their decoded spans. This avoids treating the interior bytes of an observed instruction as unexecuted data candidates. Candidate strings are then identified with printable ASCII and UTF-16LE patterns, including their terminators. Overlap with observed code prevents acceptance under this string heuristic.

Static graph recovery can itself classify a string as code. For a string that overlaps candidate graph code, the implementation requires additional structural corroboration rather than relying on printability alone. Other proposed regions originate from graph-inferred data or from bytes outside recovered blocks. The latter remain uncertain and are distinguished from stronger data candidates.

The analysis returns intervals, their kinds, and diagnostic information; it does not prove a complete partition of the payload into code and data. In particular, a byte not

observed during this run is not thereby proven unreachable or safe to move. The experimental assessment records both the proposed intervals and the limits of the run that produced them.

8.3 LLM-Based Identification

The alternative identification path presents the input through a hex view, a linear disassembly, and candidate printable runs. These are different views of the same bytes. The disassembly is explicitly heuristic, and the printable runs are candidate evidence rather than reference labels. The model returns structured data ranges, including offset and length, instead of a rewritten binary.

The parser checks that intervals have positive lengths, remain within the input bounds, and do not overlap. The engine then associates candidate ranges with its existing reference information. Ranges that cannot be accepted by the configured pipeline are reported or rejected. A fresh engine instance is used when applying a proposal, avoiding reuse of partially modified state after a rejected answer.

An invalid answer can trigger another request with feedback. The reported outcome therefore distinguishes the first proposal, later proposals, and the final accepted result. This distinction is necessary to attribute an observation correctly: a useful final extraction may reflect both the model's proposal and deterministic filtering.

The number of worked examples in this classification prompt is recorded independently of the transformation experiment. A zero-shot/two-shot comparison of assembly transformation does not imply that the same comparison has been performed for identification.

8.4 Shared Data and Reference Handling

Both identification paths connect to deterministic data-handling routines. These routines preserve the contents selected for extraction, track the references whose interpretation depends on the data layout, and finalize supported references after the code layout is known. Identification answers which regions are candidates; reference handling determines which proposed operations are supported by the implementation.

Different architectures require different addressing treatment. The x64 and x86 implementations therefore share a conceptual contract without being assumed identical at operand level. A candidate accepted in one architecture need not be accepted in the other. The evaluation keeps architecture and reference category associated with the outcome.

The pipeline also checks output integrity, including consistency of the preserved data contents and the supported references. These checks can prevent an invalid output from being accepted, but they cannot establish that all references in an arbitrary binary have been recovered. An unsupported case is consequently distinguished from a successfully handled case and from an output that is built but fails execution.

Analysis caching is a practical part of the implementation. Candidate data can be reused for an input identity, while references are recomputed against the working representation. Reproducibility requires recording whether an identification result was computed or retrieved and the configuration associated with the cached entry. Timing comparisons separate these cases.

8.5 Classification Outputs and Residual Uncertainty

Three outputs are retained for analysis: raw candidate intervals, intervals accepted by deterministic checks, and intervals ultimately processed by the data layer. Equal final binaries do not establish equal raw classifications, because filtering can remove differences. Conversely, a correct candidate can be rejected because its reference pattern is unsupported. That rejection is an integration limitation, not necessarily a classification error.

The additional evaluation in Section 11 therefore separates classification quality from downstream functionality. Successful execution supplies evidence about the observed behavior after data handling, while reference annotations supply evidence about the byte roles. Neither observation can substitute for the other.

9 LLM-Assisted Assembly Transformation

9.1 Task and Supported Transformation Families

The transformation path asks an LLM to produce a complete modified assembly representation for a selected family: EQ, NOP insertion, BCF, or IBP. These families retain their meanings from the native engine; the use of a model changes how the proposed representation is produced. Each request selects one family so that the generated artifact and its validation policy have an identifiable experimental condition.

PII and assembly wrapping remain native paths in the version examined here. They are not described as LLM-generated techniques. Likewise, a native condition combining transformations is not automatically equivalent to several model calls in sequence. The experiment identifies the implemented path rather than grouping all uses of the same family under one label.

9.2 Assembly Representation and Data Boundary

The engine prepares an assembly representation with symbolic labels for the relevant locations and preserves the data representation outside the model-generated instruction text. This preparation defines an intermediate input to the transformation task. It also determines the reference point for several structural checks and similarity measurements.

The model's text is assembled and associated with the data retained by the engine. The output is therefore a product of model generation, assembly, data handling, and validation. The experimental record retains the identity of the original file, the intermediate representation, and the final bytes, so that preparation-induced changes can be distinguished from changes in the model's output.

9.3 Zero-Shot and Two-Shot Conditions

The initial zero-shot request contains the task specification without worked examples. The initial two-shot request contains two examples in addition to that specification. The example identities and prompt versions are part of the experimental condition. Differences in their content, not only their number, can influence the comparison.

These labels apply to the first request in a run. After rejection, a later request may contain the previous answer and validation feedback. Such a run remains assigned to its initial prompting condition, but its later attempts are explicitly feedback-assisted. Reporting only the final accepted output would hide this additional interaction.

The experimental unit is a run under a specified input, transformation, prompt condition, and generation configuration. Repeated run numbers are not claimed to be independently controlled model seeds unless the service accepts and records such a parameter. Temperature, token budget, model identifier, and service configuration are recorded separately from the run index.

9.4 Validation and Feedback

Validation covers several kinds of structural evidence. Assembly must succeed, the produced code must meet the decoding checks, tracked branch destinations must have appropriate boundaries, and the expected data representation must remain consistent. Additional checks depend on the selected family and examine whether the output still contains the required instruction structure or applies a nontrivial transformation.

These checks are deliberately narrower than semantic equivalence. An output can pass structural validation yet behave differently; a conservative check can also reject an output that would have performed the expected behavior. The assessment records the reason for each rejection rather than treating all rejected answers as one category.

For supported test cases, a functional verifier can participate in the generation loop. If that verifier is also used to select accepted outputs, a later test based on the same observable is conditional on selection. The article distinguishes this pipeline acceptance criterion from an independent functional assessment and reports attempts that exhaust the allowed budget.

9.5 What Is Attributed to the Model

Three aspects are considered separately: whether an answer can be assembled, whether the complete pipeline accepts it, and whether an accepted output satisfies the reported functional check. Deterministic preparation and validation are part of the system contribution rather than capabilities attributed to the model alone.

The run record includes elapsed time, API usage when supplied by the service, rejected attempts, and the accepted artifact. Estimated cost is derived from the stated tariff snapshot; it is not presented as an invoice. Resumed jobs and reused outputs are identified so that they are not counted as additional generation calls.

10 Integration of Assembly Wrapping

10.1 Existing Technique and Contribution

Assembly wrapping is an established technique based on alternative instruction interpretations [1]. Its inclusion in this article does not claim invention of the underlying idea. The extension contributes an implementation in Zone, interaction with the engine's instruction and data representations, and a separate empirical assessment.

The relevant conceptual property is that the instruction interpretation obtained by a particular static decoding need not be the instruction stream followed during execution. This property makes it especially important to identify which representation is used by a similarity or coverage measurement. A statistic from the static view and one from the intended execution view answer different questions.

10.2 Integration and Supported Inputs

Wrapping is integrated with the native rewriting infrastructure and depends on an instruction representation adequate for the selected input. It must also interact with the final data layout and the references affected by changes in instruction placement. Construction can be refused when the required input interpretation is incomplete or an unsupported case is encountered.

Some instructions or local regions are handled differently from the main wrapped representation. The assessment therefore records the proportion represented by wrapping, retained regions, output size, construction failures, and functional outcomes. A claim about the whole output must not be inferred solely from the fraction of instructions accepted by one internal stage.

The experiment distinguishes wrapping alone, the chosen combined native condition, and the corresponding preparation-only controls. Their names identify complete configurations, including data handling and architecture. Implementation availability and successful build are reported separately from empirical functionality.

10.3 x86 Port and Development Status

The x64 implementation is available for assessment. The x86 port is under active development at the time of this draft. Register width, instruction encodings, and addressing assumptions require architecture-specific treatment, so an x64 outcome is not transferred to the x86 column of a result table.

The result layout reserves separate x86 entries. Those entries remain marked as development work until a frozen version and its assessment are available. If the port changes during a campaign, its new results are associated with the new version; they are not silently appended to results from an earlier implementation.

10.4 Assessment Boundary

Wrapping is assessed with the same separation of construction, functionality, and representation measurements used for other methods. In particular, a failure to construct an output remains part of the overall outcome. Reporting functionality only for constructed binaries can describe accepted artifacts but does not describe availability across all attempted cases. Section 11 defines both denominators.

11 Additional Experimental Methodology

11.1 Questions and Experimental Record

The additional experiments complement the original fifteen-configuration study. They examine (i) identification quality and agreement between the two code–data paths, (ii)

construction and functional outcomes after data handling, (iii) architecture-specific wrapping outcomes, and (iv) zero-shot versus two-shot LLM transformation. The original corpus and its measurements remain those of Sections 4 and 5.

Each new campaign records distinct inputs, selection criteria, architecture, configuration, repetitions, input/output hashes, and engine version or snapshot. A subset selected for low historical working rate is reported as a subset, not a new assessment of the whole corpus. Model identifier, prompt examples, generation parameters, retry policy, dependency versions, and cache status are retained where applicable. A run is distinguished from attempts within that run and from a reused artifact; run numbers do not imply controlled model seeds.

11.2 Code–Data Quality and Downstream Outcomes

Reference-based identification measures use independently annotated data-byte offsets G and predicted offsets P. With data as the positive class, precision is $|P \cap G|/|P|$, recall is $|P \cap G|/|G|$, and F_1 is their harmonic mean. Undefined cases and unresolved reference bytes are identified explicitly. Interval-level agreement complements byte scores, with the boundary or overlap rule specified in the report. Raw candidates and accepted intervals are assessed separately to distinguish classification from filtering.

Without independent annotations, the analysis is a case study of agreement and downstream functionality. Agreement with the symbolic method does not make that method ground truth, and byte-identical outputs do not establish identical raw proposals. Analysis limits, unresolved states, and computed versus cached runtimes accompany the identification outcomes.

For N planned cases, B accepted binaries, and W successful functional checks, the additional assessment reports

$$\text{BuildRate} = B/N, \qquad \text{WorkingRate}_{\text{all}} = W/N, \qquad \text{WorkingRate}_{\text{built}} = W/B. \quad (2)$$

Counts accompany percentages. Unbuilt cases remain in the planned-case denominator, and uncompleted tests remain distinct from completed failures. The observable, harness version, architecture, and observation window are stated. Connections, open ports, dialogs, and clean termination provide different empirical checks rather than a general proof of equivalence.

For LLM runs, initial acceptance and acceptance within the retry budget are separated. If the functional verifier participates in output selection, the pass rate on accepted outputs is conditional on that selection. Exhausted and restarted runs, including their time and cost, remain part of the record.

11.3 Static Verdicts, Similarity, and Cost

The published detection definition is retained for comparison. For a completed file, $d_i = m_i/a_i$ is the malicious-verdict fraction, with m_i malicious verdicts and a_i engines under the declared eligibility rule. The new report also distinguishes the fraction of completed files with any malicious verdict:

$$\overline{d} = \frac{1}{F}\sum_{i=1}^{F} d_i, \qquad q = \frac{1}{F}\sum_{i=1}^{F} \mathbf{1}[m_i > 0]. \quad (3)$$

Unsupported engines, errors, timeouts, scan dates, and pending analyses are accounted for explicitly. Pending is not zero detection. Identical hashes may reuse a scan while retaining their membership in several experimental conditions. The aggregation states whether payloads, runs, or distinct binaries receive equal weight.

The original size, entropy, and cosine measurements are retained in Section 5. The additional LLM validation score uses `SequenceMatcher` on normalized instruction signatures, not cosine similarity [21]. Its matching-block ratio is $2M/(u+v)$ for M matched elements and sequence lengths u, v. The NOP condition additionally removes recognized no-op operations; a score of one under that representation does not imply byte identity. The original file and the prepared intermediate representation are distinguished as comparison references. Any new cosine measure states its token vocabulary and whether comparisons are paired or pooled. Wrapping measurements also identify the decoding view.

Distinct output hashes characterize exact diversity independently of similarity. Size is reported as bytes and paired relative change. Timing separates analysis, preparation, API calls, and local checks where available. Costs include rejected attempts and state the tariff snapshot and estimation rule. Repetitions of the same payload are grouped for uncertainty analysis; small pilots retain case-level scope.

11.4 Validity of the Extensions

The original limitations remain relevant. Bounded exploration can miss paths because of missing environment behavior or resource limits; LLM and deterministic paths can share errors; and structural validation does not establish full semantic preservation. Changes in the harness, dependencies, input selection, or engine version can affect apparent working-rate improvements. Historical and new scans also differ in time and service conditions. Architecture-specific development, especially x86 wrapping, is associated with its own frozen version and assessment rather than inferred from x64 results.

Draft note. Finalize input counts, repetitions, model configuration, annotations, harness, scan eligibility, and aggregation from the completed campaigns. Preserve the original measurements as the published comparison.

12 Additional Experimental Results

The results below are additional to the full published study retained in Sections 4–7. New campaign outcomes will be associated with their own inputs, versions, and observation protocol. They do not silently replace the historical measurements.

12.1 Code–Data Identification and Functional Outcomes

Draft note. Insert the completed comparison, independent reference annotations if available, candidate/accepted interval results, analysis time, and functional outcomes after extraction. Keep selection and architecture explicit.

The final analysis will separate a changed classification from a changed acceptance decision or functional oracle. Any current working-rate comparison with the original study will state changes in corpus selection, preparation, and testing conditions.

Table 7. Additional code–data assessment; entries await finalized experiments. Use agreement/case-study reporting if independent labels are unavailable.

Method	Inputs	Precision_D	Recall_D	$F_{1,D}$
Symbolic/heuristic	*pending*	*pending*	*pending*	*pending*
LLM classification	*pending*	*pending*	*pending*	*pending*

Table 8. Additional wrapping outcomes. Report both planned-case and constructed-output working rates; static results refer to completed scans.

Condition	B/N	W/N	W/B	$\overline{d}$
x64 wrapping	*pending*	*pending*	*pending*	*pending*
x64 wrapping, combined	*pending*	*pending*	*pending*	*pending*
x86 wrapping	*in development*			
x86 wrapping, combined	*in development*			

12.2 Assembly Wrapping

Draft note. Insert build counts, working rates, size, diversity, and completed static scans. x86 remains development work until its version and architecture-specific campaign are fixed.

12.3 LLM Transformation: Zero-Shot versus Two-Shot

Draft note. Insert the final input/run counts, initial and final acceptance, functional outcomes, static verdicts, normalized sequence similarity, output uniqueness, attempts, time, and estimated cost. Record the scope of the pilot and the role of the generation-time verifier.

The comparison will interpret representation changes alongside construction and functional outcomes. Similarity alone does not establish correctness, and accepted outputs do not characterize the success rate of first attempts or exhausted runs.

13 Related Work

Instruction-level obfuscation has been studied in both offensive and defensive settings [26,6]. Earlier engines, including W95/Regswap [8] and MetaPHOR [24], explored register substitution and code reorganization. Instruction substitution, insertion, and control-flow transformations underpin the techniques summarized here [7,8,23,10,15].

Polymorphic encoding provides a related but distinct representation strategy. Analyses of SGN describe both the variation in its decoding stub and opportunities to recognize it [9,18]; memory-extraction research addresses the body recovered during execution [11]. For metamorphic programs, detection studies have considered Hidden Markov Models [27,8], API-trace similarity [29], and genetic-algorithm-based methods [14].

More recent work examines LLM-based assembly transformation, including the MetamorphASM benchmark and its MAD dataset [19,16]. Compiler-based systems

Table 9. Additional LLM transformation results. Similarity denotes the normalized sequence score, distinct from the historical cosine measure. Pending scans have no measured detection fraction.

Family / shots	Runs	W/N	$\overline{d}$	Similarity
EQ / 0	*pending*	*pending*	*pending*	*pending*
EQ / 2	*pending*	*pending*	*pending*	*pending*
NOP / 0	*pending*	*pending*	*pending*	*pending*
NOP / 2	*pending*	*pending*	*pending*	*pending*
BCF / 0	*pending*	*pending*	*pending*	*pending*
BCF / 2	*pending*	*pending*	*pending*	*pending*
IBP / 0	*pending*	*pending*	*pending*	*pending*
IBP / 2	*pending*	*pending*	*pending*	*pending*

such as Obfuscator-LLVM [15] offer another setting, with access to source-level or intermediate information during compilation.

The contribution of the conference paper was its payload-level *bin2bin* study, the PII design, and the joint evaluation of selected transformation configurations [4]. Those contributions are the starting point of this manuscript, rather than new claims of the extended version.

The extensions additionally connect data identification with bounded symbolic and static analysis [2], and distinguish LLM classification from assembly transformation. Assembly wrapping is attributed to its existing description and acknowledged antecedents in overlapping instructions [1]; its integration into Zone is assessed as an implementation contribution. These new directions build on the earlier payload-level study while preserving its detailed results for comparison.

14 Conclusions

Zone provides a binary-to-binary setting for studying metamorphic transformation of compact Windows payloads. The original study of 102 inputs and fifteen configurations remains the empirical foundation of this extended article. Its detection, functionality, size, entropy, and similarity results expose different effects of rewriting: lower raw-file malicious-verdict fractions coexist with substantial variation in functional outcomes and representation overhead. The PII design and the SGN comparison are retained within the scope of those published experiments.

The extension develops the code–data limitation identified by that study into an explicit identification and data-handling layer. Symbolic and heuristic analysis and an LLM-based alternative feed deterministic handling of accepted candidates. A separate LLM path supports assembly transformation with zero-shot and two-shot conditions, validation, and feedback. Integration of assembly wrapping adds an existing technique to the engine; its x86 port remains under development pending architecture-specific assessment.

The additional protocol separates classification quality, construction, working rate, static verdicts, similarity, diversity, and cost. These measurements extend rather than

replace the original evaluation. Their interpretation remains tied to the tested inputs, analysis limits, functional observables, and scan conditions. Complete semantic equivalence and general endpoint behavior require evidence beyond those observations.

Draft note. Add the principal completed extension findings here and in the abstract. Keep the original findings and update the x86 status only after its campaign is finalized.

Acknowledgments. This work was partially funded by the University of Perugia's project "Fighting Cybercrime with OSINT" (FICO), Civil Safety and Security for Society (WP2).

Disclosure of Interests. *To be completed by the authors before submission.*

References

1. 0x4ndr3: Assembly wrapping: A technique for anti-disassembly. Author's technical article, June 8 (2020), https://0x4ndr3.github.io/posts/assembly-wrapping-a-new-technique-for-anti-disassembly/, accessed 13 November 2026
2. angr Project: Control-flow graph recovery. Official angr documentation, https://docs.angr.io/en/latest/analyses/cfg.html, accessed 13 November 2026
3. Austin, T.H., Filiol, E., Josse, S., Stamp, M.: Exploring hidden markov models for virus analysis: A semantic approach. In: 2013 46th Hawaii International Conference on System Sciences. pp. 5039–5048 (2013). https://doi.org/10.1109/HICSS.2013.217
4. Bistarelli, S., Di Benedetto, J., Santini, F.: On automating the obfuscation of malware payloads: Zone, a bin2bin metamorphic engine. In: Proceedings of the 23rd International Conference on Security and Cryptography. vol. 1, pp. 216–227. SCITEPRESS (2026). https://doi.org/10.5220/0015170600004103
5. Campion, M., Dalla Preda, M., Giacobazzi, R.: Learning metamorphic malware signatures from samples. J. Comput. Virol. Hacking Tech. **17**(3), 167–183 (2021)
6. Cho, I., Towey, D., Kar, P.: Using obfuscators to test compilers: A metamorphic experience. In: 47th IEEE Annual Computers, Software, and Applications Conference, COMPSAC 2023. pp. 1786–1791. IEEE (2023)
7. Desai, P.: Towards an Undetectable Computer Virus. Master's project, San Jose State University (2008). https://doi.org/10.31979/etd.z942-squp, https://scholarworks.sjsu.edu/etd_projects/90/
8. Desai, P., Stamp, M.: A highly metamorphic virus generator. International Journal of Multimedia Intelligence and Security **1**(4), 402–427 (2010)
9. EgeBalci: Shikata Ga Nai encoder ported into Go with several improvements (2023), https://github.com/egebalci/sgn
10. es3n1n: Obfuscating native code for fun: Part 1 - introduction (2023), https://blog.es3n1n.eu/posts/obfuscator-pt-1/
11. Farley, R., Wang, X.: CodeXt: automatic extraction of obfuscated attack code from memory dump. In: Information Security: 17th International Conference, ISC 2014, Hong Kong, China, October 12-14, 2014. Proceedings 17. pp. 502–514. Springer (2014)
12. Geng, J., Wang, J., Fang, Z., Zhou, Y., Wu, D., Ge, W.: A survey of strategy-driven evasion methods for PE malware: Transformation, concealment, and attack. Comput. Secur. **137**, 103595 (2024)
13. Glynos, D.A.: Context-keyed payload encoding: Fighting the next generation of IDS. In: Athens IT Security Conference (ATH. C0N) (2010)

14. Javaheri, D., Lalbakhsh, P., Hosseinzadeh, M.: A novel method for detecting future generations of targeted and metamorphic malware based on genetic algorithm. IEEE Access **9**, 69951–69970 (2021)
15. Junod, P., Rinaldini, J., Wehrli, J., Michielin, J.: Obfuscator-LLVM: Software protection for the masses. In: 2015 IEEE/ACM 1st International Workshop on Software Protection. pp. 3–9. IEEE (2015)
16. Madani, P.: Metamorphic malware evolution: The potential and peril of large language models. In: 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications, TPS-ISA 2023, Atlanta, GA, USA, November 1-4, 2023. pp. 74–81. IEEE (2023)
17. Madenur Sridhara, S., Stamp, M.: Metamorphic worm that carries its own morphing engine. J. Comput. Virol. **9**(2), 49–58 (2013)
18. Miller, S., Reese, E., Carr, N.: Shikata Ga Nai encoder still going strong (2019), https://cloud.google.com/blog/topics/threat-intelligence/shikata-ga-nai-encoder-still-going-strong/
19. Mohseni, S., Mohammadi, S., Tilwani, D., Saxena, Y., Ndawula, G.K., Vema, S., Raff, E., Gaur, M.: Can LLMs obfuscate code? a systematic analysis of large language models into assembly code obfuscation. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 39, pp. 24893–24901. AAAI Press (2025). https://doi.org/10.1609/aaai.v39i23.34672
20. Nawaz, M.S., Fournier-Viger, P., Nawaz, M.Z., Chen, G., Wu, Y.: MalSPM: Metamorphic malware behavior analysis and classification using sequential pattern mining. Comput. Secur. **118**, 102741 (2022)
21. Python Software Foundation: difflib: Helpers for computing deltas. Python documentation, https://docs.python.org/3/library/difflib.html, accessed 13 November 2026
22. Sathyanarayan, V.S., Kohli, P., Bruhadeshwar, B.: Signature generation and detection of malware families. In: Information Security and Privacy: 13th Australasian Conference, ACISP 2008, Wollongong, Australia, July 7-9, 2008. Proceedings 13. pp. 336–349. Springer (2008)
23. Szor, P., Ferrie, P.: Hunting for metamorphic. In: Virus bulletin conference (2001)
24. The Mental Driller: Metamorphic permutating high-obfuscating reassembler (2002), http://dsr.segfault.es/stuff/website-mirrors/29A/29a-6/29a-6.602
25. Venkatesan, A.: Code Obfuscation and Virus Detection. Master's project, San Jose State University (2008), https://scholarworks.sjsu.edu/etd_projects/116/
26. Vishwas, G., Nithin, N.S., Varshith, P., Belwal, M.: Unveiling the world of code obfuscation: A comprehensive survey. In: 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS). pp. 1–8. IEEE (2023)
27. Wong, W., Stamp, M.: Hunting for metamorphic engines. Journal in Computer Virology **2**, 211–229 (2006)
28. Zeng, D., Tan, G.: From debugging-information based binary-level type inference to CFG generation. In: Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy. pp. 366–376. CODASPY '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3176258.3176309
29. Zhang, Q., Reeves, D.S.: MetaAware: Identifying metamorphic malware. In: Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007). pp. 411–420. IEEE (2007)